
Mathlib as a Geometry of Mathematics: An Empirical Study of Formal Proof Graphs

Ege Çakar¹
Harvard University

Abstract

Large formal libraries make it possible to study mathematics as an explicit combinatorial object and to ask where automated theorem-proving agents may succeed or fail. This paper analyzes Mathlib through two graphs extracted from Lean-Dojo traces: a theorem-dependency graph, whose edges record premise use, and a state-tactic hypergraph, whose edges record goal-level proof-state transformations induced by tactics. These graphs are used to test whether the formal structure of Mathlib has measurable geometry, whether that geometry predicts theorem-level quantities, and whether graph paths can approximate mathematical reduction from one theorem to another. Mathlib’s dependency graph has strong large-scale structure: a heavy-tailed degree distribution, 240 Louvain communities with modularity 0.592, mostly linear proof trees, and predominantly negative sampled Ollivier-Ricci curvature. Graph features also predict proof length: degree alone gives test $R^2 = 0.483$, while adding a node2vec random-walk embedding raises performance to $R^2 = 0.555$; in contrast, normalized-adjacency eigenvectors give little useful signal across the tested category, link, tactic, and proof-length tasks. Finally, state-graph pathfinding under strict goal-text equality recovers only 2.5% of literal dependency edges, suggesting that graph geometry is useful as measurement infrastructure for AI theorem proving, but that discovery tools need canonicalized proof-state matching and tactic-internal transformations rather than surface graph paths alone.

1. Introduction

Mathematical practice has always relied on informal geometric language: subjects are adjacent or distant, methods travel across fields, some lemmas are central, and some definitions create bottlenecks through which later arguments must pass. Formalized mathematics turns part of this metaphor into data. In a library such as Mathlib, theorems cite other theorems, tactics transform goals into subgoals, and proof states record the intermediate objects that connect a statement to its proof.

This paper studies that data as a geometry of mathematics. The goal is not to claim that a dependency graph captures mathematical meaning in full. It clearly does not. Rather, the question is whether the graph-theoretic shadow of formal mathematics carries useful statistical information for theorem-proving systems. For AI mathematical agents, this is a measurement question: before a system can search, prove, or propose abstractions inside a formal library, it helps to know which parts of the library are central, which local neighborhoods predict proof complexity, and which graph relations fail to capture actual proof transformations.

Four concrete questions guide the analysis:

1. What is the large-scale structure of Mathlib’s theorem-dependency graph?
2. Do graph-geometric features predict theorem-level quantities such as proof length or top-level Mathlib area?
3. Are spectral coordinates a useful representation of this graph?
4. Can proof-state paths make precise the idea that theorem B reduces to theorem A ?

The main positive result is that local graph structure is predictive. Degree is already a strong baseline for proof length, and node2vec embeddings add information beyond degree. The main negative result is equally useful: straightforward spectral coordinates of the normalized adjacency are weak

¹Harvard University. Correspondence to: Ege Çakar <ecakar@college.harvard.edu>.

features for the tested prediction tasks. The pathfinding experiments show a second limitation. A theorem may depend on another theorem without ever having that theorem’s statement appear as a literal proof goal, so any discovery tool based on paths must canonicalize goals and match subterms rather than compare printed goal text directly.

Contributions. This paper makes three empirical contributions. First, it constructs a theorem-dependency graph and a state-tactic hypergraph from LeanDojo traces of Mathlib, separating premise-use structure from proof-state transformation structure. Second, it evaluates several graph-geometric representations on concrete prediction tasks, finding that local random-walk geometry predicts proof length while naive spectral coordinates are weak in this setting. Third, it tests a literal proof-state path notion of theorem reduction and shows that it is too brittle for theorem discovery, thereby identifying where future proof-transformation graphs should expose tactic-internal operations such as rewriting, simplification, instantiation, and elaboration.

2. Background and Related Work

Mathlib. Mathlib is a community-developed library of formal mathematics in the Lean theorem prover. The Mathlib paper describes the library’s dependent type theoretic foundations, emphasis on classical mathematics, hierarchy of structures, automation, and distributed organization ([The mathlib Community, 2019](#)). These design choices make Mathlib both mathematically broad and structurally analyzable: proofs are machine-checkable objects, and the library’s declarations form a large explicit network of dependencies.

LeanDojo and premise annotations. LeanDojo introduced an open environment, dataset, and benchmark for machine learning on Lean proofs ([Yang et al., 2023](#)). Its traces include fine-grained premise annotations: for a tactic in a proof, the dataset records which accessible declarations were used. Those annotations are the source of the certified theorem-dependency graph used here. LeanDojo also introduced ReProver, a retrieval-augmented prover, illustrating how the existing mathlib graph, and perhaps dependency structure can be used operationally in theorem proving.

The structure of mathematics. Recent work has explicitly raised the question of whether AI systems can help reveal the global structure of mathematics. Barkeshli, Douglas, and Freedman describe mathematics in terms of universal proof and structural hypergraphs and argue that AI may make it possible to study the terrain of formal proofs at scale ([Barkeshli et al., 2026](#)). That work is conceptual: it proposes graph-theoretic language for the structure of mathematics, but does not primarily analyze a concrete existing

proof graph. The present paper is an empirical counterpart, built from Mathlib traces.

Network analysis of Mathlib. Closest in spirit is recent work on the network structure of Mathlib, which extracts a multilayer graph with 308,129 declarations, 8.4 million edges, and 7,563 modules ([Li et al., 2026](#)). That study emphasizes the distinction between explicit, compiler-synthesized, and proof-driven dependencies, and finds that namespace taxonomies and logical structure can diverge. The present analysis focuses on a different slice of the library and a different goal: LeanDojo premise edges, proof-state hyperedges, and prediction tasks designed to test which graph features carry theorem-level signal.

3. Data and Graph Construction

The analysis uses LeanDojo Benchmark 4 traces. Two graph objects are constructed from the same proof corpus. A LeanDojo theorem record contains a formal declaration name, its source file, and a sequence of traced tactic invocations. A tactic invocation includes the tactic string, the proof state before the tactic, the proof state after the tactic, and a list of premises used by the tactic. A premise is any accessible declaration that LeanDojo attributes to that tactic step: it may be a theorem, lemma, definition, or other named declaration. Top-level Mathlib area labels are obtained from the first directory under `Mathlib/`, such as `Algebra`, `Topology`, or `MeasureTheory`.

3.1. The Theorem-Dependency Graph

The theorem-dependency graph is a directed graph $G_{\text{dep}} = (V, E)$. Its vertices are named declarations that appear either as traced theorems or as premises used by traced theorems. There is an edge

$$B \rightarrow A$$

when some traced tactic in the proof of declaration B uses declaration A as a premise. The edge direction therefore points from the theorem being proved to one of its ingredients. Following outgoing edges walks backward from a conclusion toward the facts used to prove it; following incoming edges finds later results that reuse a given declaration.

If B ’s proof mentions A multiple times, those mentions are collapsed to one unique edge for graph statistics, while an occurrence count is retained as metadata. This distinction matters: unique edges measure the shape of the dependency graph, whereas premise occurrences measure repeated use inside proof scripts.

Each premise-use record stores the source theorem, target premise, source and target file paths, Mathlib area labels, tactic index, tactic text, and tactic head. For example, an

Table 1. Theorem-dependency graph counts.

Quantity	Value
Theorems indexed	122,517
Theorems with traced tactics	61,544
Tactic invocations	259,580
Premise occurrences	418,704
Unique dependency edges	309,396
Nodes	138,333
Weakly connected components	44,479
Largest weak component	92,594

edge can record that theorem B ’s fourth tactic was `rw [A]`, or that a later tactic was `apply A`. This makes dependency paths checkable against the trace annotations: a path $B \rightarrow C \rightarrow A$ is not an inferred semantic relation, but a chain of observed premise uses in Mathlib proofs.

3.2. The State-Tactic Hypergraph

The dependency graph records which theorems are used; it does not record how a goal changes during proof search. The state-tactic hypergraph fills that gap. Its vertices are Lean tactic states, represented by the printed local context and target goal. For instance, a state includes the local hypotheses currently available to the proof and the proposition still to be shown.

A raw LeanDojo tactic invocation may contain several active goals in its `state_before` field. The state-tactic graph therefore uses *goal-level* transitions rather than raw tactic rows as its basic unit. After splitting printed multi-goal states into single-goal states, a tactic row with several active input goals can contribute several graph edges, one for each active input goal represented in that row. Thus an edge in this graph should be read as a goal-local tactic transition, not as a one-to-one encoding of a LeanDojo tactic invocation.

A goal-level tactic transition is represented as a directed hyperedge

$$s \xrightarrow{\tau} \{s_1, \dots, s_k\},$$

where s is one active input goal, τ is the tactic string, and $\{s_1, \dots, s_k\}$ is the set of single-goal output states associated with that branch after applying the tactic. The case $k = 0$ means the tactic closed the goal. The case $k = 1$ is an ordinary single-goal transition. The case $k > 1$ is a branch: the tactic split the selected goal into several subgoals, all of which must eventually be solved.

This construction makes subsequent tactic steps easier to connect to the corresponding branch, but it remains syntactic: it does not unfold definitions or identify propositionally equivalent goals.

For the comparison with non-Mathlib Lean code, the paper

Table 2. State-tactic hypergraph counts.

Quantity	Value
Unique single-goal states	261,655
Unique goal-level tactic hyperedges	261,107
Multi-output hyperedges	18,467
Indexed traced theorems	61,544

Note. Because the graph is constructed after splitting multi-goal input states, the number of goal-level tactic hyperedges can exceed the number of raw LeanDojo tactic invocations reported in Table 1.

also uses a Lean-GitHub corpus: a broader collection of Lean proofs mined from public GitHub repositories. Unlike Mathlib, this corpus includes miscellaneous projects outside a single curated library, so it should be interpreted as a contrast with less standardized Lean code rather than as a matched collection of equally polished mathematical developments.

The state-tactic hypergraph is closer to proof search, but it is also more syntactic: two goals that are mathematically equivalent may have different printed forms. A state-graph path from theorem B toward theorem A therefore means that the proof of B reaches a proof state whose printed target matches the initial target of A . This is stricter than ordinary theorem use in Lean, where A may be used by rewriting, simplification, instantiation, or term-mode construction without ever appearing as the current goal. There are also difficulties that arise from how Lean handles certain shorthand tactics.

4. Methods

Features. The predictive experiments use a small set of graph-derived feature families, all meant to be a reasonable embedding one might use if they believe geometry holds information.

Degree. The baseline graph features are log in-degree and log out-degree in the theorem-dependency graph. Out-degree measures the number of distinct premises used in a proof; in-degree measures how often a theorem is reused.

Spectral Features and Node2Vec. Spectral features are computed on the undirected version of the dependency graph. If A is the symmetrized adjacency matrix and D is the degree matrix, the normalized adjacency is

$$\hat{A} = D^{-1/2} A D^{-1/2}.$$

We compute 32 nontrivial leading eigenvectors of $\hat{A}$ and 32 eigenvectors from the opposite end of the spectrum, equivalently high-frequency directions for the normalized Laplacian. These are stored as `low_1, ..., low_32` and

high₁, ..., high₃₂. The proof-length spectral baseline uses the first 10 low and first 10 high coordinates. The link and tactic-head endpoint models use all 64 stored spectral coordinates, together with endpoint degree features where specified. The category-prediction experiments include low-frequency, high-frequency, and mixed low/high variants with 10 and 32 spectral coordinates, both with and without degree features. We also test a mixed 5+5 variant using the first five low-frequency and first five high-frequency coordinates. The reported headline spectral-plus-degree category result uses the first ten low-frequency coordinates together with degree features.

Node2vec features are trained on the undirected dependency graph using unbiased random walks, i.e. $p = q = 1$. The run used embedding dimension $d = 64$, 10 walks per node, walk length 40, window size 5, 5 negative samples, skip-gram Word2Vec training, 3 epochs, and seed 0.

Centrality, resistance, and curvature. Additional geometric summaries include approximate betweenness centrality, approximate effective resistance to high-indegree hubs, and sampled Ollivier-Ricci curvature on dependency edges.

Prediction task details. All supervised experiments use fixed seed 0 and deterministic 80/20 train/test splits. For node-level tasks, a declaration is assigned to the training set when a stable hash bucket of its name is less than 8 out of 10; otherwise it is assigned to the test set. For edge and pair tasks, the same hashing rule is applied to the ordered pair key. This keeps the train/test split fixed across feature families within each task.

Proof-length regression is evaluated on held-out traced theorem nodes with target $\log(1 + \text{tactic count})$. The proof-length models are ridge regressions on standardized features: degree features use $\log(1 + \text{out-degree})$ and $\log(1 + \text{in-degree})$; the node2vec and combined graph-geometry regressions add the corresponding embedding, betweenness, and resistance features. Link prediction uses 10,000 observed dependency edges as positives and an equal number of sampled non-edges as negatives, excluding self-pairs and already observed dependency edges in either direction; it is evaluated with binary logistic regression and held-out AUC. Tactic-head prediction uses observed dependency edges whose tactic head is one of the eight most common heads

```
{rw, simp, have, exact, refine,
  simp_rw, simp_a, apply},
```

and is evaluated with multiclass logistic regression. Top-level Mathlib area prediction is evaluated with held-out accuracy and macro-F1 using softmax logistic regression and a

Table 3. Large-scale structural statistics for Mathlib.

Structural statistic	Value
Power-law exponent for degree tail	2.370
Tail threshold $x_{\min}$	43
Louvain communities	240
Louvain modularity	0.592
Mean proof-tree depth	2.815
Median proof-tree depth	2
Maximum proof-tree depth	38

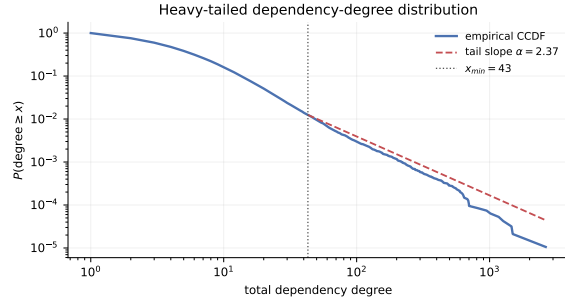


Figure 1. Empirical complementary CDF of total dependency degree. The dashed line shows the fitted power-law tail above $x_{\min} = 43$.

one-hidden-layer MLP with hidden dimension 128; the baselines are majority class, degree-only, and the namespace-prefix prior.

5. Results

5.1. Mathlib Has Strong Large-Scale Graph Structure

The dependency graph is far from an unstructured theorem list. Its degree distribution has a heavy tail; fitting the tail gives exponent 2.370 above $x_{\min} = 43$. Louvain community detection gives 240 communities with modularity 0.592. Proof trees are shallow on average, with mean depth 2.815, median depth 2, and maximum depth 38.

The heavy tail is visible at the level of total dependency degree, counting unique incoming and outgoing dependency edges incident to each declaration. Among positive-degree declarations, the median total degree is 3, the 90th percentile is 13, the 99th percentile is 48, and the maximum is 2,645. Thus, most declarations participate in only a few premise relations, while a small set of central declarations acts as reusable infrastructure.

The proof-shape distribution is dominated by linear proofs: 50,191 of 61,544 traced proofs are classified as linear. Mathlib also differs from the broader Lean-GitHub corpus. Mathlib proofs are shorter and use a smaller tactic vocabulary. Since Lean-GitHub contains assorted repositories rather

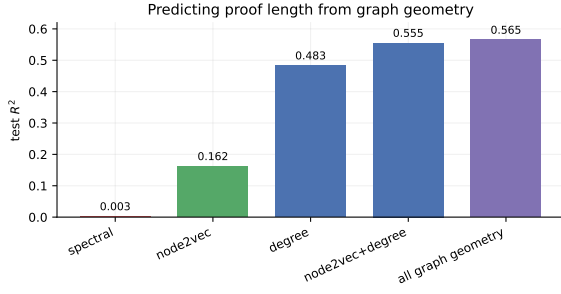


Figure 2. Proof-length prediction from graph features. Spectral coordinates are weak, while node2vec adds useful local-neighborhood signal on top of degree.

than only curated Mathlib code, the length discrepancy should be read primarily as a curation and style effect, not as a controlled comparison of theorem difficulty.

5.2. Degree and Local Neighborhoods Predict Proof Length

Proof length is the clearest positive predictive task. Degree alone gives test $R^2 = 0.483$ for predicting $\log(1 + \text{tactic count})$. This result reflects an intuitive phenomenon: theorems whose proofs cite more distinct premises tend to have longer proofs.

The more interesting comparison is between graph embeddings. Spectral coordinates alone have almost no proof-length signal ($R^2 = 0.003$). A node2vec embedding alone reaches $R^2 = 0.162$, and node2vec plus degree raises performance from $R^2 = 0.483$ to $R^2 = 0.555$. Combining degree, node2vec, betweenness, and resistance gives $R^2 = 0.565$.

The conclusion is that local graph neighborhoods carry information beyond raw degree, but the naive spectral encoding does not capture that information well. Variants using both the highest and lowest eigenvalue eigenvectors, while keeping the total number of spectral coordinates, did not improve performance. Because out-degree and tactic count both increase with proof elaboration, the degree result should be interpreted as a strong but not surprising baseline. The node2vec improvement is more informative: it suggests that nearby proof dependencies carry additional signal about proof complexity.

5.3. Spectral Coordinates Are Weak Predictors

Top-level Mathlib area prediction gives the sharpest negative result. The majority baseline has accuracy 0.165. The best spectral-plus-degree model reaches 0.177. A namespace-prefix prior (using the top level namespace for the theorem) reaches 0.737, showing that much of the area label is encoded in naming conventions rather than in the tested spec-

Table 4. Summary of the spectral prediction experiments.

Task	Degree	Spectral	Interpretation
Link AUC	0.875	0.881	small gain
Tactic-head acc.	0.349	0.346	no gain
Proof-length R^2	0.483	0.003	fails alone

tral coordinates.

This should be read narrowly. The dependency graph is predictive under other representations, especially node2vec. What fails is the simplest eigenvector-coordinate representation of a heavy-tailed dependency graph. However, it does tell us the strength of information stored in basic spectral features is relatively weak.

5.4. Sampled Curvature Suggests Tree-Like Local Structure

Ollivier-Ricci curvature was estimated on 3,320 sampled dependency edges by comparing one-hop neighborhoods across each edge. The mean sampled curvature is -0.781 , the median is -0.840 , and 88.7% of evaluated edges have negative curvature. The curvature result is consistent with the degree and proof-tree statistics: Mathlib’s dependency graph is sparse, hub-driven, and locally tree-like rather than triangle-rich and Euclidean.

5.5. Dependency Paths Are Certified, but Proof-State Paths Are Brittle

The theorem-dependency graph supports certified dependency paths. Each path edge is backed by a LeanDojo premise annotation at a concrete tactic step, so a path $B \rightarrow \dots \rightarrow A$ can be read as an observed chain of premise uses inside Mathlib proofs rather than as an inferred semantic similarity relation.

The state-tactic graph asks a stricter question: does a proof state in the proof of B literally become the initial goal of theorem A ? Under this textual goal-matching relation, random theorem pairs are almost never connected. With maximum search depth 12, only 2 of 2,000 random theorem pairs were connected. Among $N = 1,000$ evaluated dependency edges $B \rightarrow A$, where the dependency graph already records that B cites A , strict goal-text matching recovered 25 edges, or 2.5%.

This is not a contradiction. “Uses theorem A ” and “has A ’s statement as the current goal” are different relations. Rewriting, simplification, implicit arguments, term-mode proof construction, and theorem instantiation all create dependency edges without producing literal goal equality. A useful path-based discovery system therefore needs goal canonicalization and subterm matching, especially for `rw`

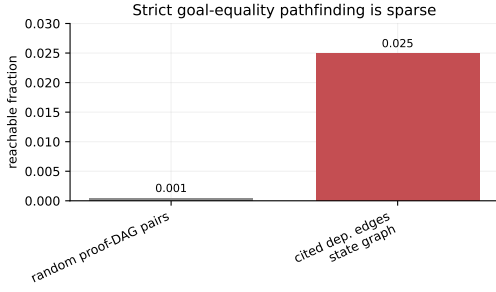


Figure 3. Strict goal-equality pathfinding is sparse. The 2.5% dep-edge rate is measured on pairs where the dependency graph already records that one theorem cites the other.

and `simp` uses. This failure mode is informative: it shows that dependency traces are useful measurement objects, but that theorem-discovery systems must represent how tactics transform terms rather than only which declarations appear in a proof.

5.6. Repeated Tactic Idioms Suggest Abstraction Candidates

Tactic-sequence compression gives a lightweight way to identify repeated proof idioms. A BPE-style analysis of tactic heads finds the dominant macro

```
rw -> [0] -> exact,
```

with 7,895 occurrences. Other high-frequency patterns include `have -> [0] -> rw`, `ext -> [0] -> simp`, `have -> [0] -> have`, and `rw -> [0] -> simp`, where `[0]` denotes the branch marker in the extracted tactic-token representation. These patterns are best interpreted as general Lean proof-engineering idioms. At the tactic-head level, `rw`, `simp`, and `exact` mostly record rewriting, simplification, and goal-closing operations while hiding the theorem schemas, matched subterms, instantiated arguments, and side conditions that made the step work. The analysis therefore identifies where repeated proof behavior occurs, but the abstraction-search problem requires the finer proof-transformation graph described below.

6. Toward Proof-Transformation Graphs

The state graph should be expanded from tactic-level transitions into the underlying proof-transformation graph on which tactics operate. The main object to unpack is not the tactic head itself, but the sequence of semantic operations hidden by automation. For `simp`, this means exposing the selected `simp` lemmas, rewrite directions, matched subterms, instantiated variables, congruence steps, generated side conditions, and recursive simplification subgoals. For `rw`, the step is less a search procedure than a specified rewrite; the useful expansion is therefore the matched occurrence, di-

rection, instantiated theorem schema, and side goals. For `exact`, the relevant operation is type-checking and elaboration of a proposed term against the target, so the graph should expose the elaborated proof term and unification constraints rather than treat `exact` as an abstraction candidate. The following graph is proposed as a target representation for future abstraction search rather than as an object fully constructed here.

A lower-level graph would have nodes for proof states, subterms, instantiated theorem schemas, generated side goals, elaborated proof terms, and unification constraints. Edges would record primitive transformations such as rewriting a matched subterm, discharging a side condition, applying congruence, or checking that a term inhabits the current target. New theorem search could then be posed as verified graph compression: find repeated subgraphs across unrelated proofs, synthesize a candidate lemma or theorem schema that summarizes the repeated transformation, replace the repeated subgraph by a call to that candidate, and count it only if Lean verifies the new declaration and the modified proofs. This is the natural version of abstraction discovery for Mathlib, because it searches over the proof operations that tactics actually performed rather than over surface-level tactic n-grams.

7. Limitations

The main limitations are representational. First, dependency edges identify premise use but do not by themselves explain how a premise is used. Second, state-graph paths currently use printed goal equality, which misses equivalent goals, instantiated theorem statements, and rewrite-style uses. Third, the curvature calculation is sampled rather than exhaustive. Fourth, proof length is an imperfect difficulty proxy: it mixes mathematical difficulty, proof-engineering style, library availability, and tactic automation. With the assumption that Mathlib proofs are close to optimal, this is a useful first step, but not the end. Even when proof length is useful as a first proxy, it should not be treated as a direct measure of intrinsic mathematical difficulty.

A second limitation is causal interpretation. The predictive experiments show that graph features correlate with theorem-level quantities, not that those features cause proof difficulty. For example, out-degree and tactic count are mechanically related because longer proofs have more opportunities to cite distinct premises. The more cautious interpretation is that graph geometry provides useful measurement features for theorem-proving systems, not that it fully explains mathematical complexity.

8. Conclusion

Mathlib’s formal proof corpus supports a concrete empirical study of the geometry of mathematics. The dependency graph has clear macroscopic structure, local graph neighborhoods predict proof length, and sampled curvature suggests a sparse tree-like organization around hubs and bottlenecks. At the same time, the most straightforward spectral features are weak, and literal proof-state pathfinding is too brittle to serve as theorem discovery.

The resulting picture is not that graph geometry solves theorem discovery, but that it gives a measurable substrate for asking sharper questions. The next step is to connect this substrate to verified prover performance and to refine the state graph until common proof transformations become visible below the level of opaque tactic calls.

We do not believe this is the end to working with mathematical graphs like Mathlib, but hope that it is a useful starting place.

Software and Data

Code for constructing the theorem-dependency graph and state-tactic hypergraph from LeanDojo Benchmark 4 traces over Mathlib, computing the main graph features, and reproducing the reported prediction and pathfinding experiments is available at https://github.com/Ege-Cakar/mathlib_analysis. The raw LeanDojo traces are not redistributed with this code; the code assumes that the user has obtained the corresponding public traces and points the configuration file to their local copy.

References

- Barkeshli, M., Douglas, M. R., and Freedman, M. H. Artificial intelligence and the structure of mathematics. *arXiv preprint arXiv:2604.06107*, 2026. URL <https://arxiv.org/abs/2604.06107>.
- Li, X., Peng, N., Severini, S., and Shafto, P. The network structure of Mathlib. *arXiv preprint arXiv:2604.24797*, 2026. URL <https://arxiv.org/abs/2604.24797>.
- The mathlib Community. The Lean mathematical library. *arXiv preprint arXiv:1910.09336*, 2019. URL <https://arxiv.org/abs/1910.09336>.
- Yang, K., Swope, A. M., Gu, A., Chalamala, R., Song, P., Yu, S., Godil, S., Prenger, R., and Anandkumar, A. LeanDojo: Theorem proving with retrieval-augmented language models. *arXiv preprint arXiv:2306.15626*, 2023. URL <https://arxiv.org/abs/2306.15626>.